Technical introduction to PSOC™ Edge E84 features

Chapter 5: SMIF

About this document

Scope and purpose

This document is the training manual for the technical introduction to PSOC™ Edge E84 features. This manual helps in getting started with different applications leveraging key features for PSOC™ Edge, using ModusToolbox™ software.

The training is divided into multiple chapters covering topics like audio, graphics, machine learning, among others.

Chapter 5 discusses PSOC™ Edge's SMIF and external memory support.

Intended audience

The document is intended for design engineers, technicians, and developers of electronic systems.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Prerequisites **4**
3 Code example: Serial flash read/write **5**
3.1 Objective 5
3.2 Description 5
3.3 Application flow 5
3.4 Hardware overview 7
3.5 Project creation 7
3.6 Output 9
3.7 Modify the application 9
3.7.1 Change the external flash address 9
3.7.2 Place code or data at a desired memory location in external flash 11
3.8 Conclusion 15
4 Code example: PSRAM read and write in XIP mode **16**
4.1 Objective 16
4.2 Description 16
4.3 Application flow 16
4.4 Hardware overview 17
4.5 Project creation 18
4.6 Output 21
4.7 Modify the application 21
4.7.1 Enable caching 21
4.7.2 Placing an array to PSRAM 23
4.8 Conclusion 26
5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ **27**
6 Appendix B: KIT_PSE84_EVAL details **30**
Revision history **31**
Disclaimer **32**

1 Introduction

This manual provides detailed instructions to create, configure, build, and run several application code examples on the PSOC™ Edge E84 MCU.

2 Prerequisites

See the **Prerequisites** section in the **Chapter 1 training manual** for a complete list of prerequisites and development tools.

3 Code example: Serial flash read/write

3.1 Objective

This code example demonstrates how to interface with and perform read/write/erase operations on an external NOR flash memory, using the serial flash library with the PSOC™ Edge E84 MCU.

3.2 Description

This code example shows how the serial memory interface (SMIF) block in the PSOC™ Edge E84 MCU is used to access an external flash memory device.

The SMIF block in the PSOC™ Edge E84 MCU supports accessing memory in two modes:

- MMIO (Memory-Mapped I/O) mode: This mode accesses memory using offset addresses (assuming a start address of 0x0000 0000). To read data from memory, a serial-flash API function must be used
- XIP (eXecute-In-Place) mode: This mode directly accesses memory using MCU addresses (the start address is as mentioned in the linker). No API is required to read data from memory. The data can be accessed through direct addresses mapped to the CPU (for example, 0x6034 0400)

This code example mainly focuses on reading, writing, and erasing operations using MMIO mode.

In addition, this code example covers how to use the Serial Flash Discoverable Parameters (SFDP) standard, which allows for auto-discovery of flash parameters, as well as commands for read, program, and erase operations.

3.3 Application flow

The example has a two-CPU (Arm® Cortex® M33 CPU (CM33) and Arm® Cortex® M55 CPU (CM55)), three-project structure, including CM33 Secure, CM33 Non-Secure, and CM55 projects. All three projects are programmed to the external QSPI flash and executed in XIP mode.

This example uses a SMIF hardware block for interfacing with the external memory through a SPI interface, with four data lines and one slave select line. This example writes 64-bytes of data to the end sector of external memory. The written data is read back to check its integrity.

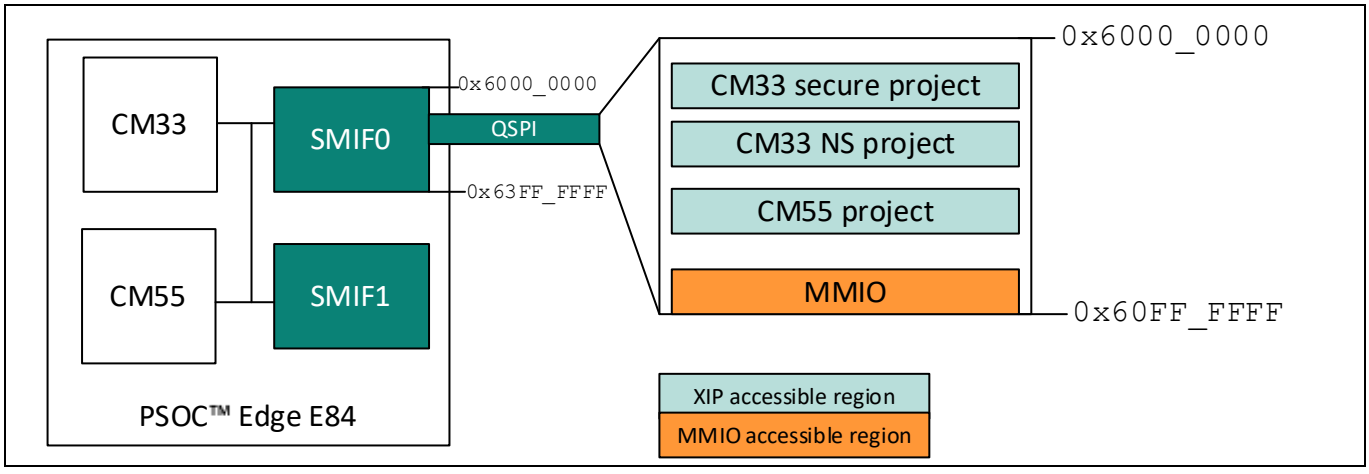


Figure 1 Memory implementation of serial flash code example

Code example: Serial flash read/write

This example uses the serial-memory library along with a memory configuration generated using the **QSPI Configurator** tool. This tool can be accessed from the **Quick Panel** in Eclipse IDE for ModusToolbox™. This tool generates the configuration files *cycfg_qspi_memslot.c/.h* under the directory *bsps/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/GeneratedSource/* based on the memory part selected.

By default, the example comes with a fixed configuration of the **Memory Part Number** (MPN) used in the BSP. But you can change it by selecting the MPN of your choice from a dropdown under the Memory Part Number column.

The following figure shows the default configuration for the example used in this lab:

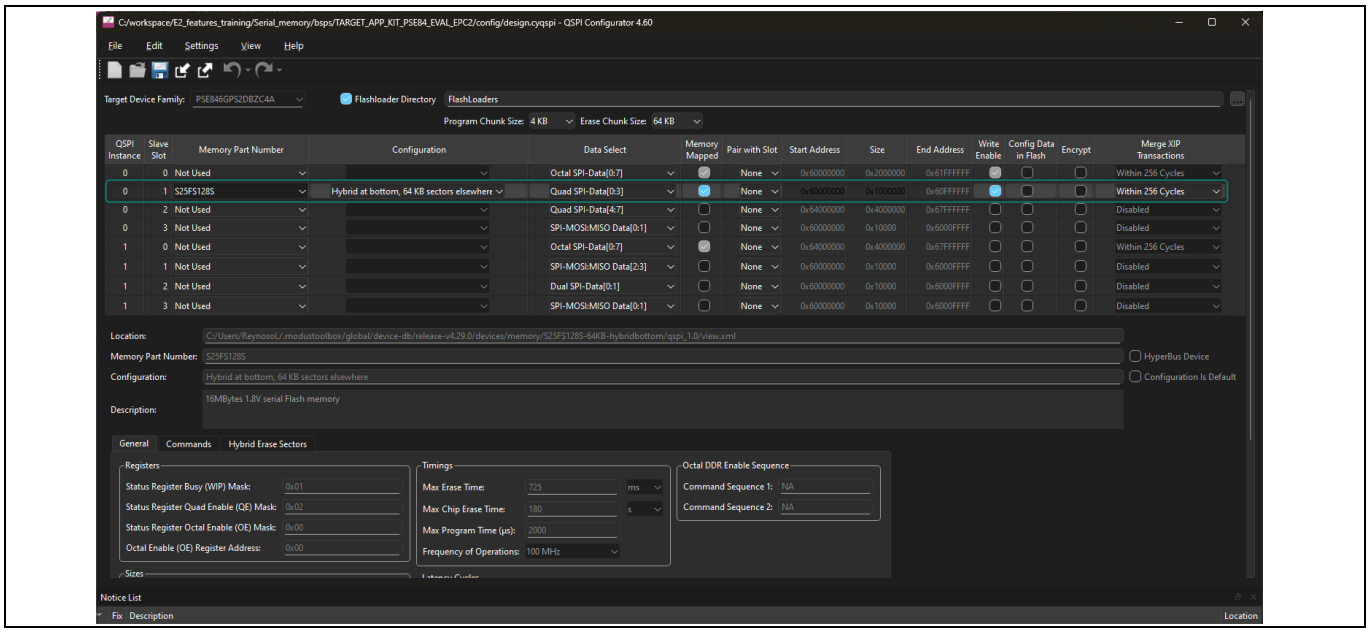


Figure 2 QSPI Configurator for Serial flash example

Observe the following fields:

- **QSPI Instance**: This memory is using SMIF0
- **Slave slot**: Using SMIF0_select1
- **Memory Part Number**: Example is using S25FS128S QSPI flash included in SOM. Other memories are supported by the QSPI Configurator, or developers can use the SFDP feature
- **Configuration:** For some memories, there's additional configuration information, such as 3-byte or 4-byte addressing mode
- **Data select:** Selects the data lines for the corresponding slot. This memory is implemented in QSPI configuration
- **Memory Mapped:** Enables XIP mode
- **Pair with Slot:** Determines the paired slot for dual quad operation. Not used in this case
- **Start Address/Size/End Address:** This memory is placed from 0x6000_0000-0x60FF_FFFF
- **Write Enable**: Write access is enabled
- **Config Data in Flash**: Determines whether a specific memory slot's configuration structures are to be placed in Flash or SRAM. This configuration uses SRAM
- **Encrypt:** Encryption is not used by default on this memory
- **Merge XIP Transactions**: Specifies how many cycles can pass between memory accesses while still skipping the overhead of re-sending commands

3.4 Hardware overview

SMIF0:

- SMIF0.CLKP
- P2.0/SPIF0.select1
- P1[0:3]/SMIF0[0:3]

Note: This example requires BOOT SW in ON position.

See

Appendix B: KIT_PSE84_EVAL details for more details.

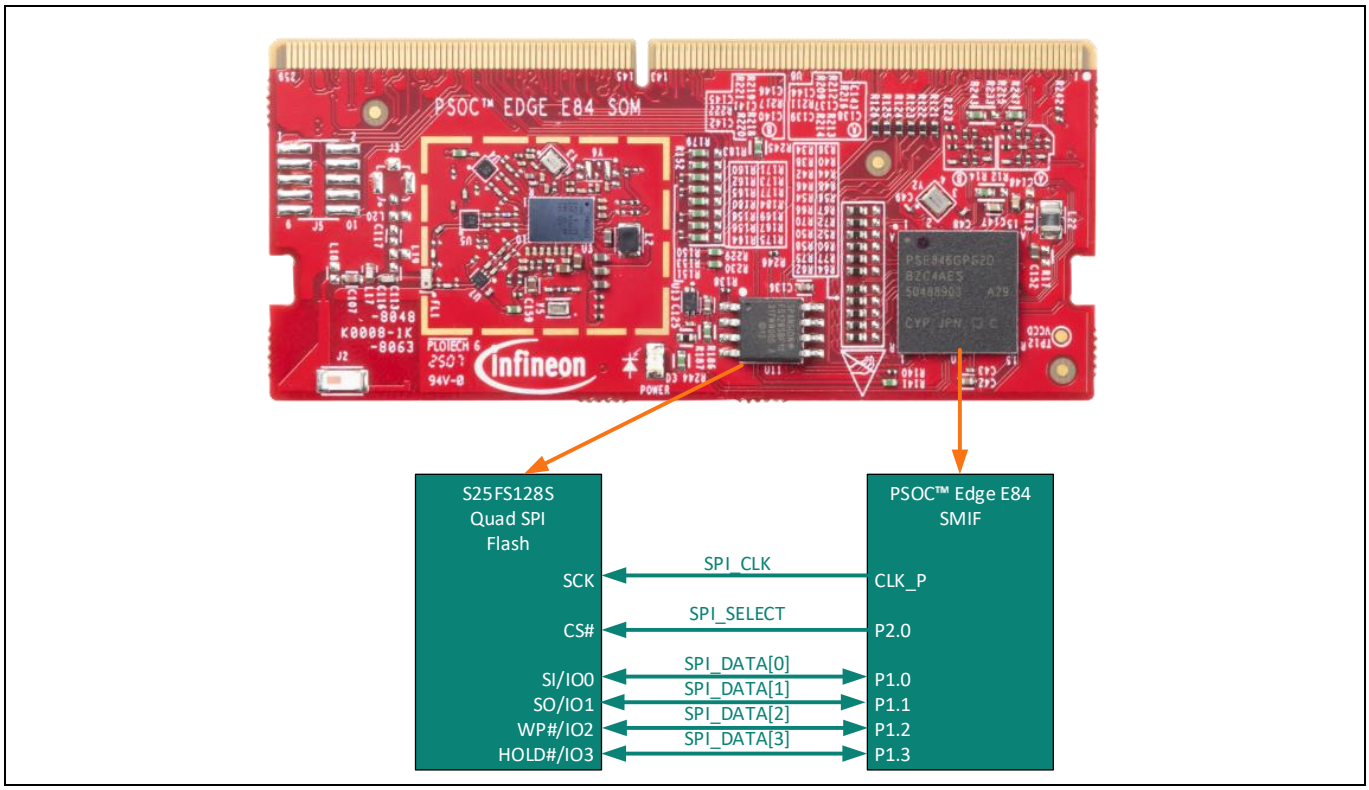


Figure 3 Hardware diagram for Serial flash example

3.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge Serial Flash Read and Write and XIP** application under the **Peripherals** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

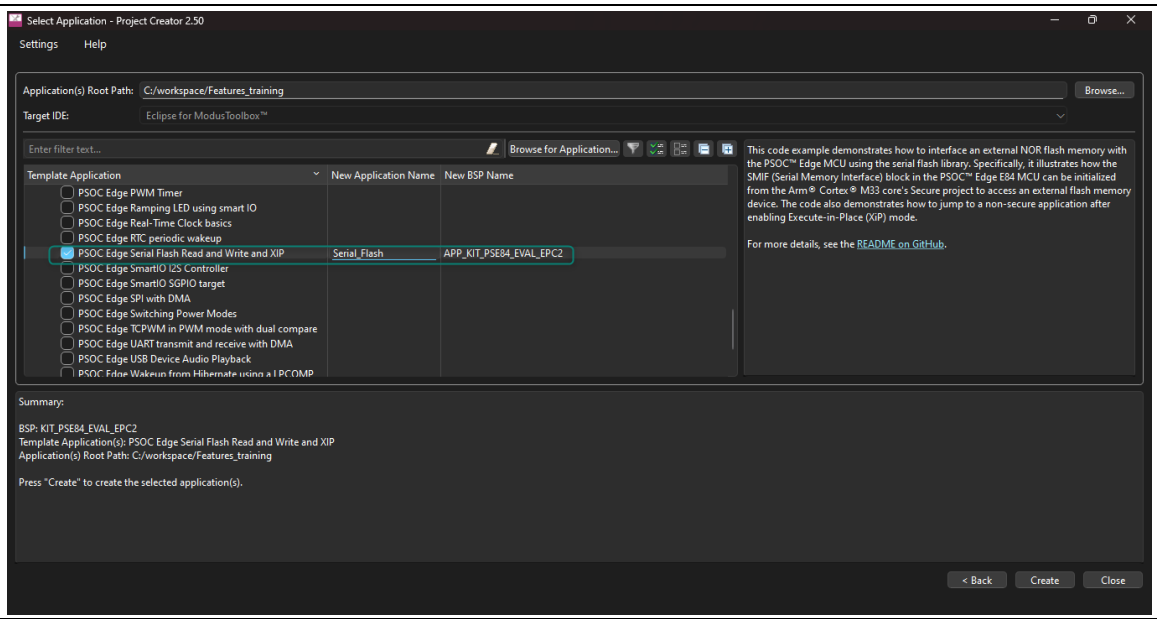


Figure 4 Creating serial flash read and write and XIP example

3. **Note**: This code example utilizes the **serial-memory** library.
   Open the **Library Manager** from the Quick Panel, and observe the library being included for the cm33_ns project. You can observe other libraries included in the project, like mtb-dsl-pse8xxgp support, CMSIS, and so on.

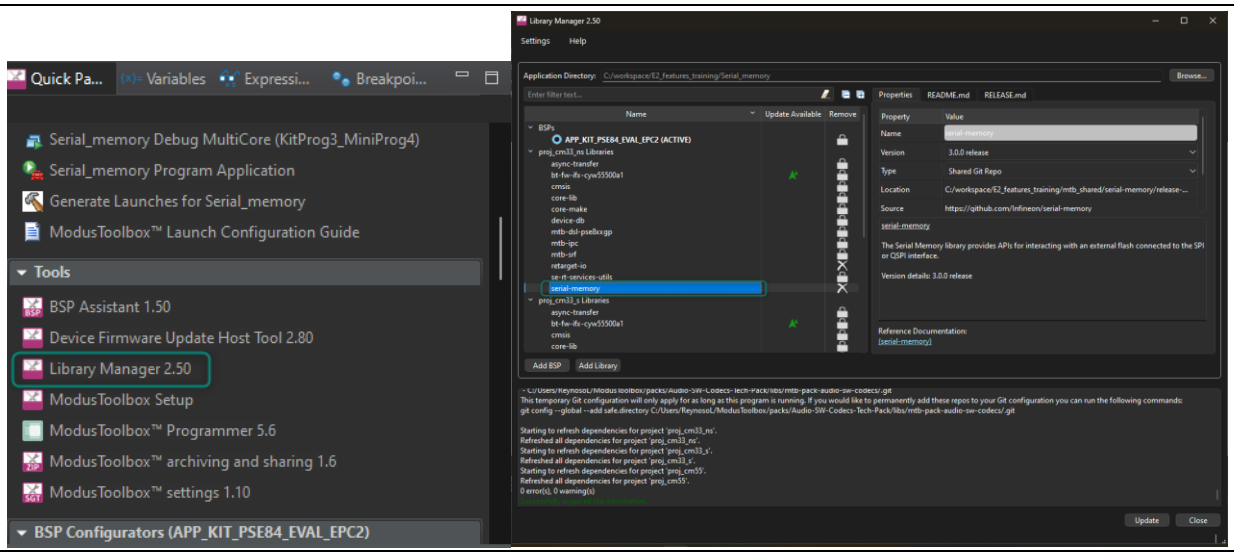


Figure 5 Library Manager configuration for serial flash example

4. **Note**: We will not be performing modifications before running the default code example, but feel free to open *proj_cm33_ns/main.c* to analyze the application flow.
   Observe the implementation of the following APIs, which are documented in the serial memory API guide:
   - **mtb_serial_memory_setup**: initializes and sets up the serial memory
   - **mtb_serial_memory_erase**: the application erases memory using this API
   - **mtb_serial_memory_read**: the application reads memory to confirm that memory is erased and programmed correctly
   - **mtb_serial_memory_write**: the application uses this API to write a buffer to memory

5. Build and program the application.

3.6 Output

After successful programming, the application starts automatically. Confirm that the UART terminal displays the following terminal output:

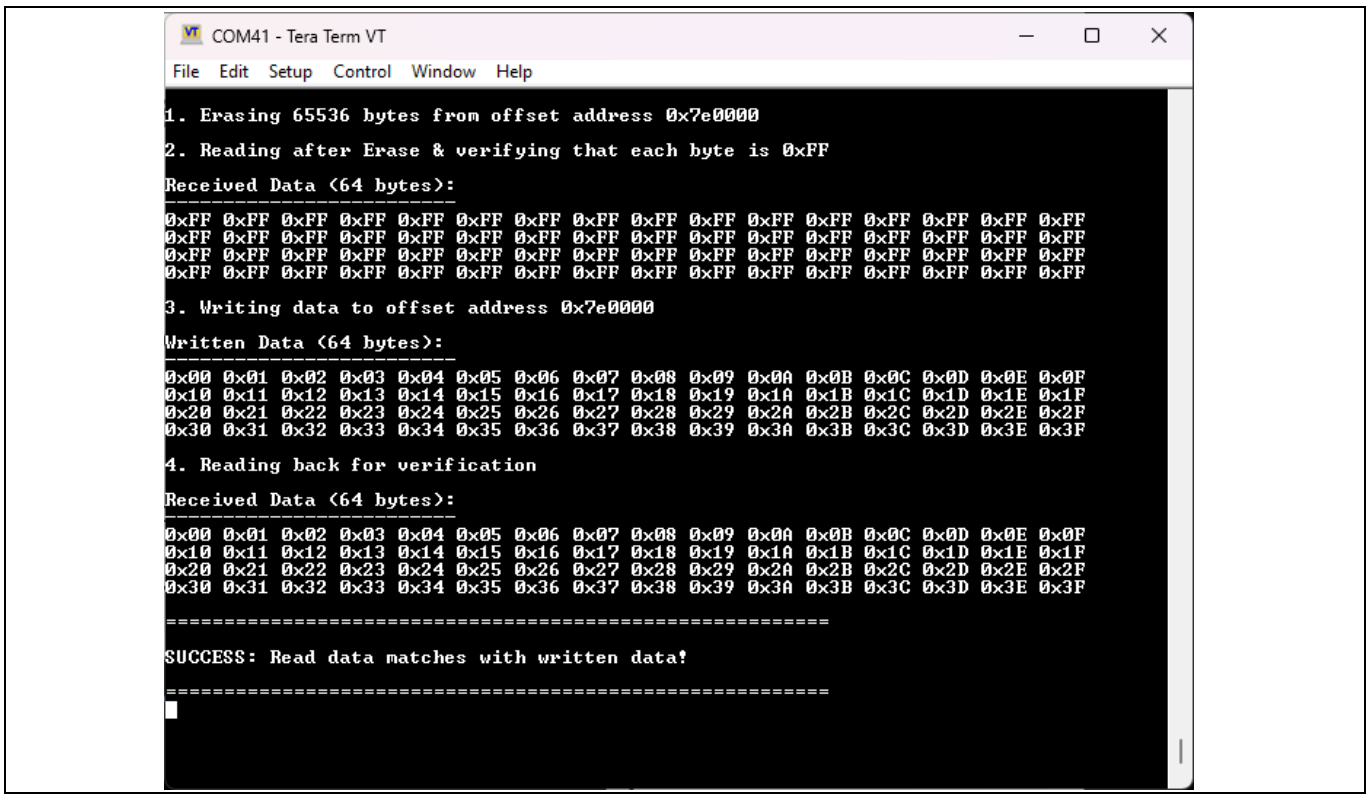


Figure 6 Serial flash example output

3.7 Modify the application

3.7.1 Change the external flash address

Try changing the default address used for the read / write operation.

Hints for the advanced modification:

- Check the APIs used by the application and modify the address parameter to use a different location

Solution

1. **Note**: The application uses the variable *ext_mem_address* to define the address used by the application, and by default, it simply uses the last sector which corresponds to address 0x7E_0000 as observed in the application output.
   Note: This is an offset, so the mapped address really corresponds to 0x607E_0000.

Code example: Serial flash read/write

Code listing 1 Default assignment of ext_mem_address

```
/* Use last sector to erase for flash operation */
ext_mem_address = (smifMemConfigs[MEM_SLOT_NUM]->deviceCfg->memSize/
MEM_SLOT_DIVIDER - smifMemConfigs[MEM_SLOT_NUM]->deviceCfg->eraseSize *
MEM_SLOT_MULTIPLIER);
```

2. In *proj_cm33_ns/main.c*, replace the default assignment of *ext_mem_address,* and write to 0xB00000, which corresponds to mapped location 0x60B0_0000.

Code listing 2 Modification to main.c to change the address

```
check_status("Serial memory setup failed", result);

/* Use last sector to erase for flash operation */
//ext_mem_address = (smifMemConfigs[MEM_SLOT_NUM]->deviceCfg->memSize/
//                   MEM_SLOT_DIVIDER -
//                   smifMemConfigs[MEM_SLOT_NUM]->deviceCfg->eraseSize *
//                   MEM_SLOT_MULTIPLIER);

/* Write address 0x60B0_0000 */
ext_mem_address = 0xB00000;

sectorSize = mtb_serial_memory_get_erase_size(&serial_memory_obj, ext_mem_address);
```

3. Build and program the modified application.
   The terminal should show the corresponding modification.

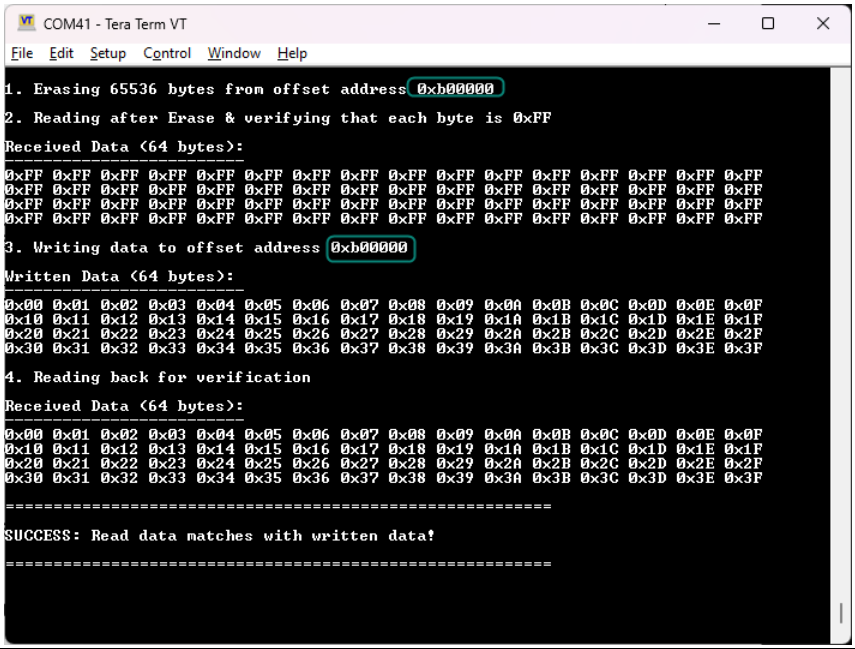


Figure 7 Output of serial flash example after changing the address

3.7.2 Place code or data at a desired memory location in external flash

Try modifying the application to place a code or data to a specific memory location in external flash

Hints for the advanced modification

- Create new sections for code and data using Memory Configurator
- Define the sections in the linker file
- Place code and data using CY_SECTION

Solution

1. Open **Device Configurator** from the Quick Panel.
2. Select the **Memory** tab and expand the SMIF block 0, memory 1.

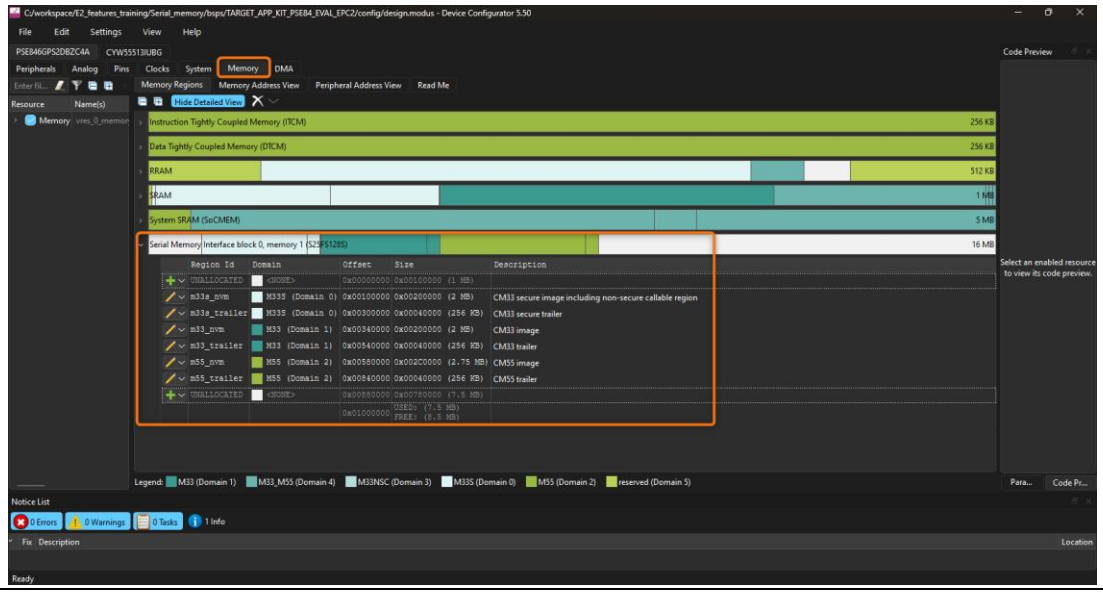


Figure 8 Default configuration in Memory Configurator

3. Click the plus sign on the last region and select "**Add region**"
4. Type "custom" as **Region id**, select "M33 (Domain 1)" as **Domain**, and define the **Offset** and **Size** as "0x00A00000" and "0x00100000" respectively.

   Note: This will create a custom 1 MB region accessible for the non-secure CM33 core from 0x60A00000-0x60AFFFFF.

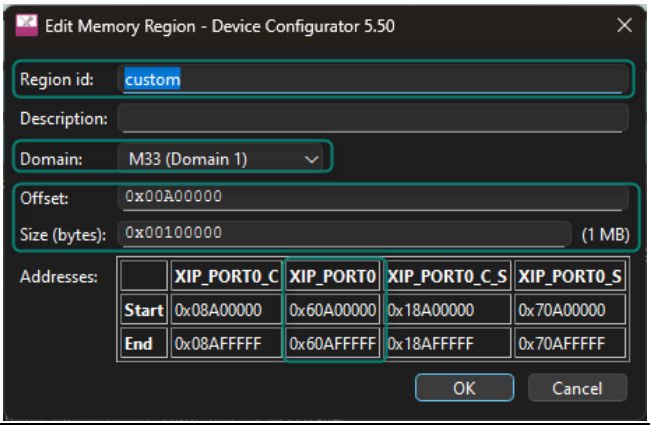


Figure 9 Adding a custom section to Memory Configurator

5. Click **OK** to save the new region. The Memory Regions will update as shown below:

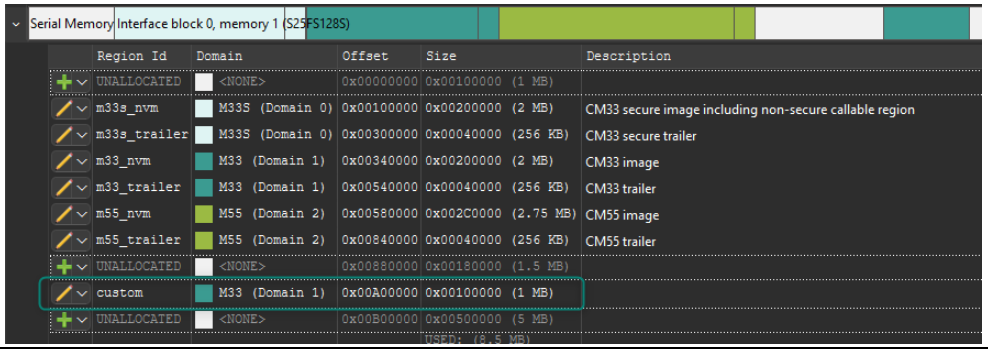


Figure 10 New custom section

6. Note: Open the **Memory Address View** to observe the new section in the memory map.

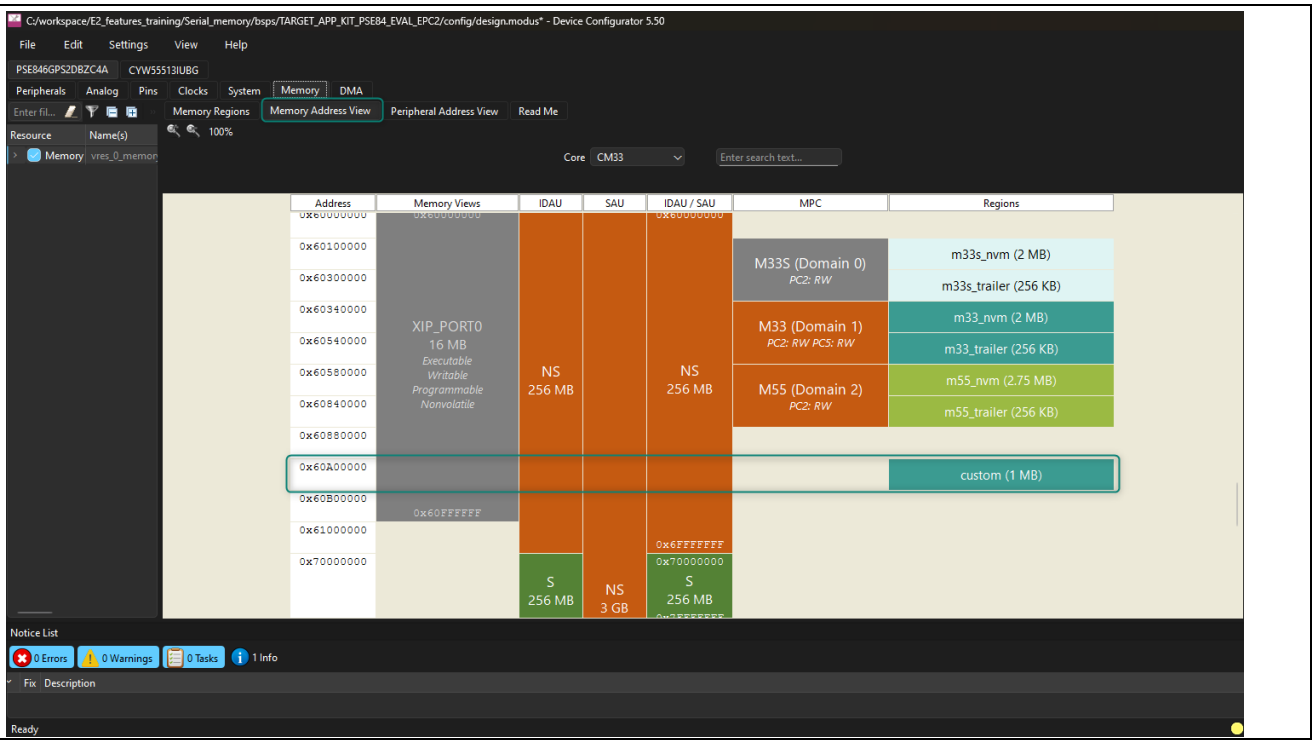


Figure 11 Updated memory map

7. Save and close the device configurator. This will generate updated files for our application.
8. Open the GCC linker file located at *<project_path>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\COMPONENT_CM33\TOOLCHAIN_GCC_ARM\pse84_ns_cm33.ld*

9. Add new sections for custom code and data as shown in the following code fragment.

Code listing 3 Adding new sections in linker file

```
/* Define the sections */
SECTIONS
{
    .custom_code_section :
    {
        KEEP(*(.custom_code_section))
    } > custom

    .custom_const_section :
    {
        KEEP(*(.custom_const_section))
    } > custom

    /* This section reserves a space for MCUBoot header */
```

10. In *proj_cm33_ns/main.c,* add the following code to add a variable and a function in the new sections and to print their locations:

Code listing 4 Declaring a variable and function in new sections

```
/*******************************************************************************
 * Global Variables
 *******************************************************************************/
/* Objects for serial memory middleware */
static mtb_serial_memory_t serial_memory_obj;
static cy_stc_smif_mem_context_t smif_mem_context;
static cy_stc_smif_mem_info_t smif_mem_info;
CY_SECTION(".custom_const_section") __attribute__((used)) const char hi_word[] =
"Hello string from the custom data section!\n";


/*******************************************************************************
 * Function Definitions
 *******************************************************************************/
CY_SECTION(".custom_code_section") __attribute__((used)) void
print_from_custom_address(const char *buf);


void print_from_custom_address(const char *buf)
{
    printf("%s", buf);
}
```

11. Then add the following code to print the location of the variable and function:

Code listing 5 Printing the location of custom variable and function

```
/* Check if the transmitted and received arrays are equal */
check_status("Read data does not match with written data. Read/Write "
        "operation failed.", memcmp(tx_buf, rx_buf, PACKET_SIZE));

/* Advanced Modification Start */
uint32_t addr = (uint32_t)&hi_word;
/* Print the string from external memory */
printf("\nString in the external memory at address: 0x%08"PRIx32"\n", addr);
printf("\n-----------------------------------------------------\n%s", (char *)
&hi_word);

addr = (uint32_t)&print_from_custom_address;
/* Print by calling function that lives in external memory */
printf("\nFunction call from external memory address: 0x%08"PRIx32"\n", addr);
printf("\n-----------------------------------------------------");
print_from_custom_address("\nHello from function stored in the custom code
section!\n");
/* Advanced Modification End */

printf("\r\n=====================================================\r\n");
printf("\r\nSUCCESS: Read data matches with written data!\r\n");
printf("\r\n=====================================================\r\n");
```

12. Rebuild and reprogram the application. Observe that the new function and constant data are placed in the custom region located from 0x60A0_0000 to 0x60AF_FFFF as shown in Figure 12.

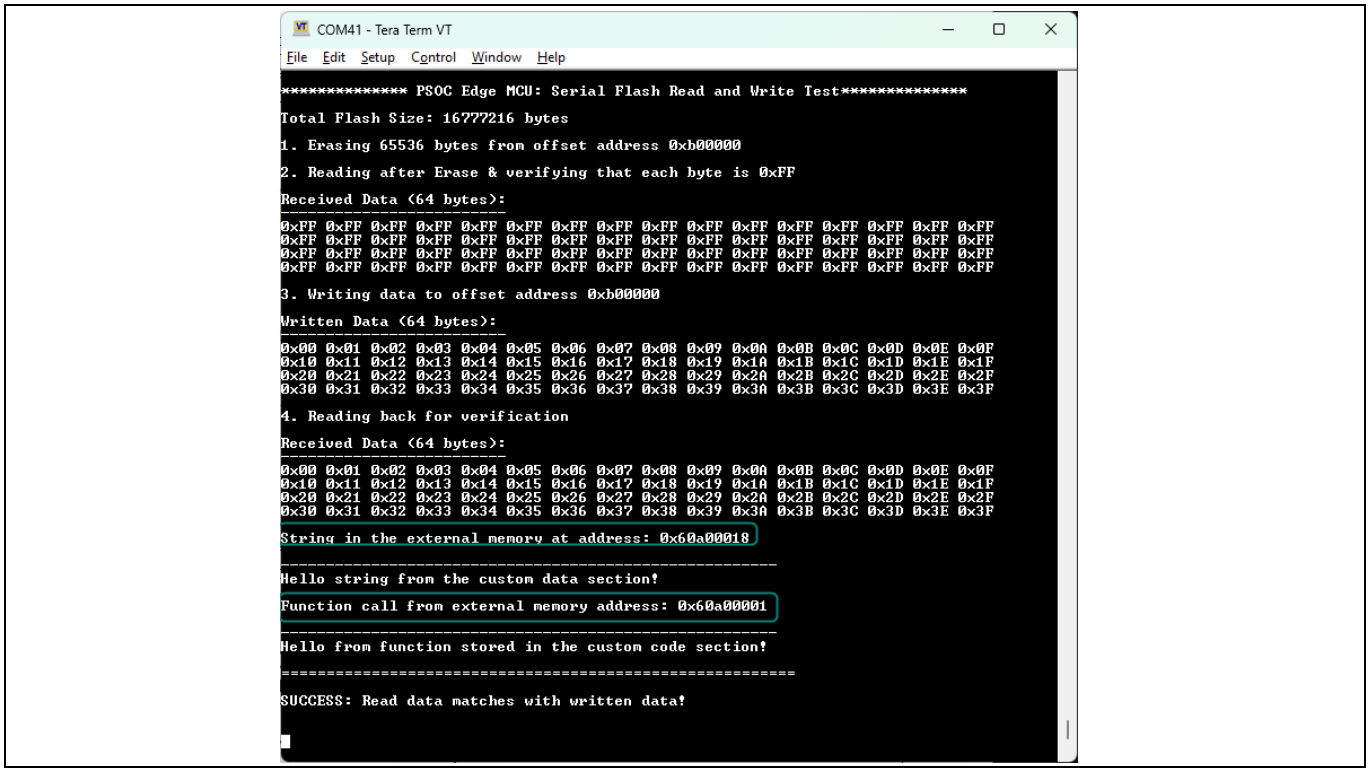


Figure 12 Output after placing a variable and function in custom memory

3.8 Conclusion

We have successfully enabled the PSOC™ Edge E84 MCU to interface with memory using the SMIF block and perform erase, read, and write operations using serial flash, in ModusToolbox™.

4 Code example: PSRAM read and write in XIP mode

4.1 Objective

This code example demonstrates how to interface an external PSRAM with the serial memory interface (SMIF) in the PSOC™ Edge E84 MCU. The example also shows how to perform read and write operations while in eXecute-In-Place (XIP) mode.

4.2 Description

This code example uses an external PSRAM interfaced with the SMIF1 block in the PSOC™ Edge E84 MCU. First, the example initializes the SMIF1 block using serial-flash API functions. Next, it initializes the memory slot for the PSRAM based on the memory configuration done inside the QSPI configurator using PDL API functions.

An SMIF block supports accessing memory in two modes:

- MMIO (Memory-Mapped I/O) mode (**normal mode**): This mode accesses memory using offset addresses (assuming a start address of 0x0000 0000). To read / write data from memory, serial-flash API functions must be used
- XIP (eXecute-In-Place) mode (**memory mode**): This mode directly accesses memory using MCU addresses (the start address is as mentioned in the linker). No API is required to read data from memory. The data can be accessed through direct addresses mapped to the CPU (for example, 0x6034 0400)

This code example mainly focuses on read and write operations using XIP mode.

Then, the example configures the SMIF1 block to enter XIP mode to use the PSRAM as extended memory.

Finally, the example performs read and write operations on the PSRAM to demonstrate its functionality.

4.3 Application flow

The example has a two-CPU (Arm® Cortex® M33 CPU (CM33) and Arm® Cortex® M55 CPU (CM55)), three-project structure, including CM33 Secure, CM33 Non-Secure, and CM55 projects. All three projects are programmed to the external QSPI flash interfaced to SMIF0 and executed in XIP mode.

The CM33 Non-Secure project code starts with initializing clock and system resources in the BSP initialization function. Then, Retarget-IO is initialized to enable print logs for the example's output. Next, the SMIF1 HW block and memory slot are initialized to communicate with the external PSRAM.

After setting up the memory interface, the example sends a write enable command to the PSRAM, enabling write operations. Then, SMIF1 is set to XIP mode, allowing the PSOC™ Edge E84 MCU to execute code directly from the external PSRAM.

Finally, the example can perform read and write operations by directly pointing to PSRAM addresses mapped to the CPU.

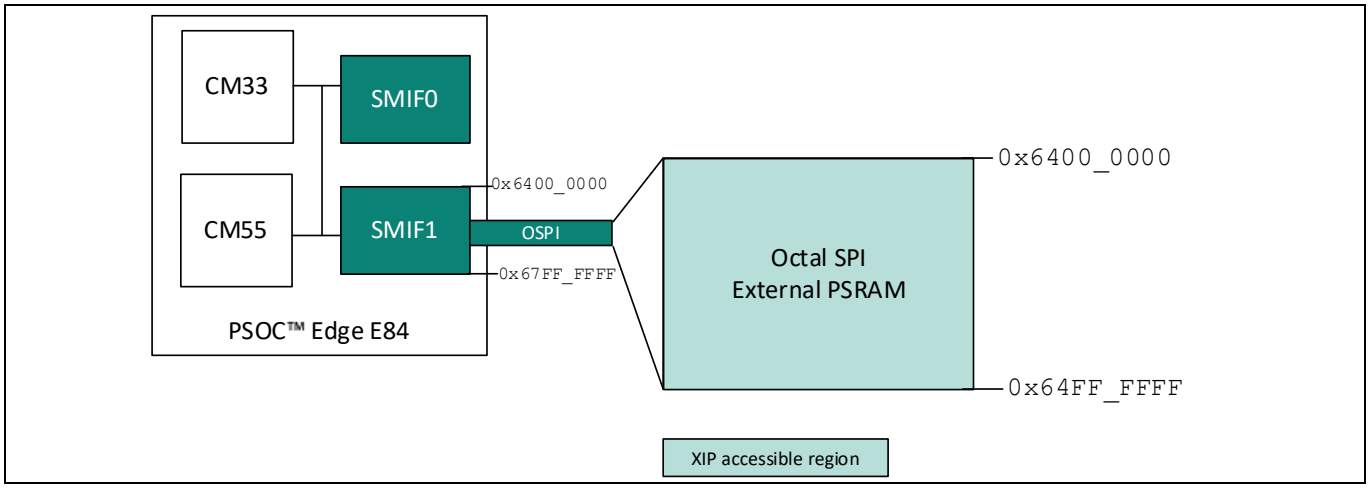


Figure 13 Memory configuration for PSRAM example

4.4 Hardware overview

SMIF1:

- SMIF1.CLKP
- SMIF1.RWDS
- P0.1/SMIF1.select2
- P4[0:7]/SMIF1[0:7]
- P21.5/MEMORY_RST

Note: This example requires BOOT SW in ON position.

See Appendix B: KIT_PSE84_EVAL details for more details.

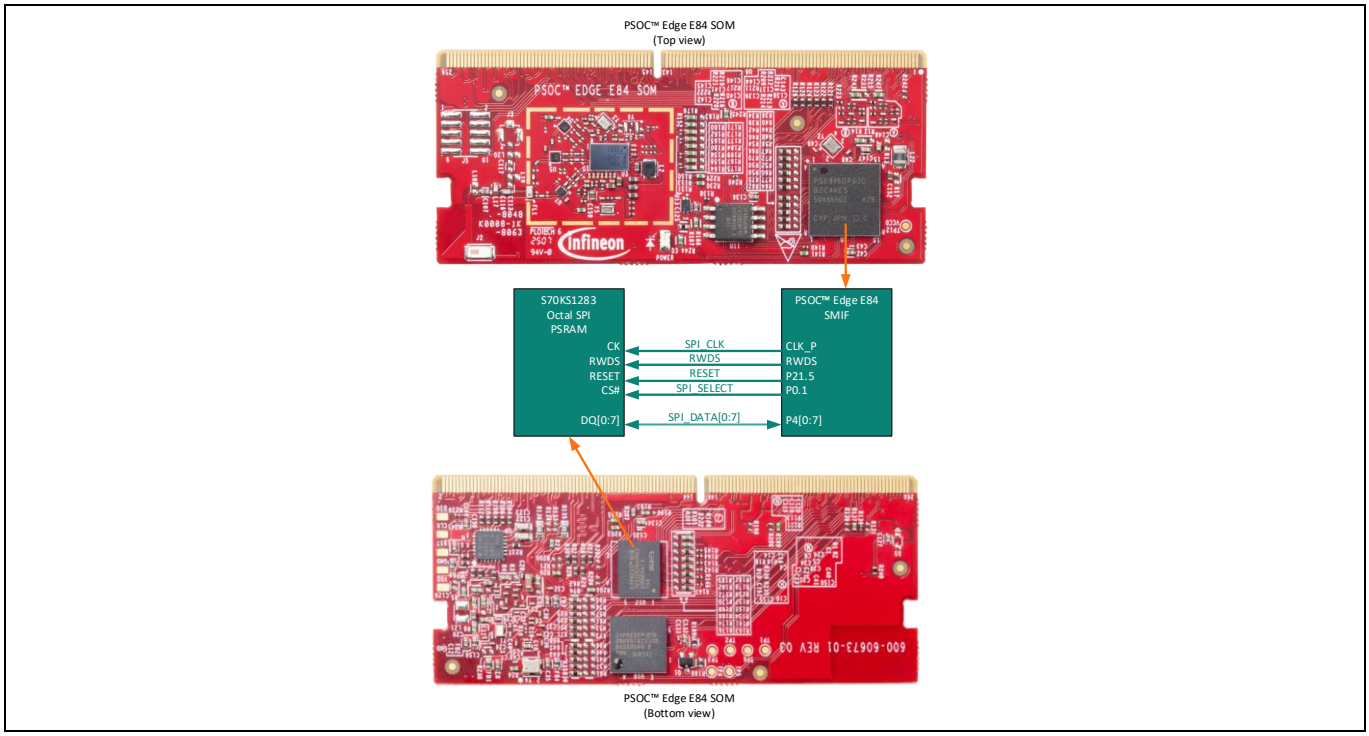


Figure 14 Hardware diagram of PSRAM read and write example

4.5 Project creation

1. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application.
2. When creating the application, select the **PSOC™ Edge PSRAM Read and Write in XIP mode** application under the **Peripherals** section.

   Note: To prevent issues with Windows path length, it's recommended to rename the project if your worskpace path is long.

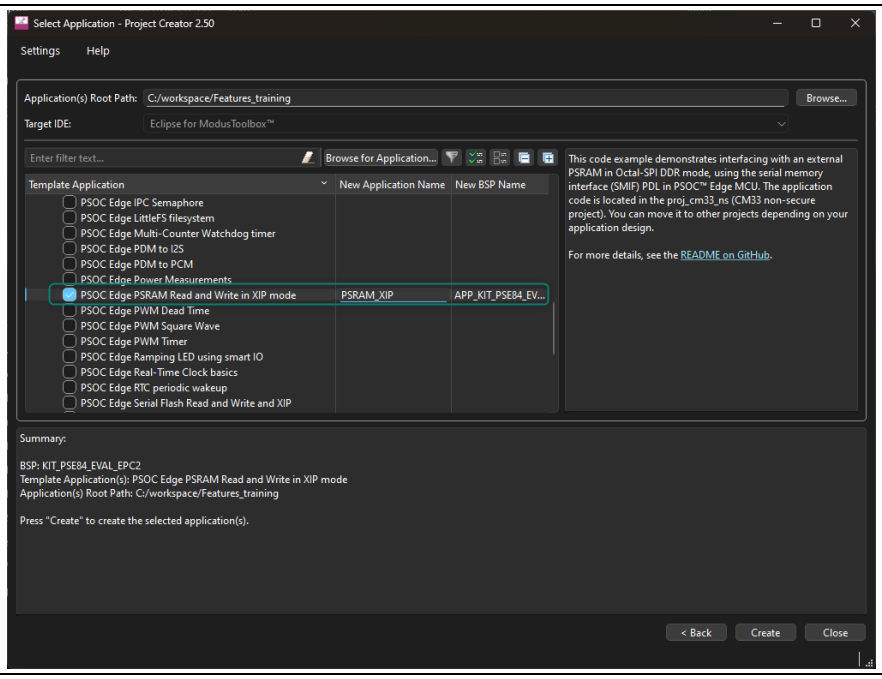


Figure 15 Creating PSRAM read and write in XIP mode example

3. **Note**: The PSRAM example provides custom configurations to interface with the PSRAM in octal mode, which are stored in the ***design.cyqspi*** files under the templates directory. This file is copied to the BSPs directory during project creation and replaces the default configuration of the KIT_PSOCE84_EVK BSP. This updated configuration is necessary to ensure proper communication with the external PSRAM and enable the SMIF1 block to operate in octal mode.

   Open the **QSPI Configurator** from the Quick Panel and observe the configuration of the octal SPI memory highlighted below:

Code example: PSRAM read and write in XIP mode

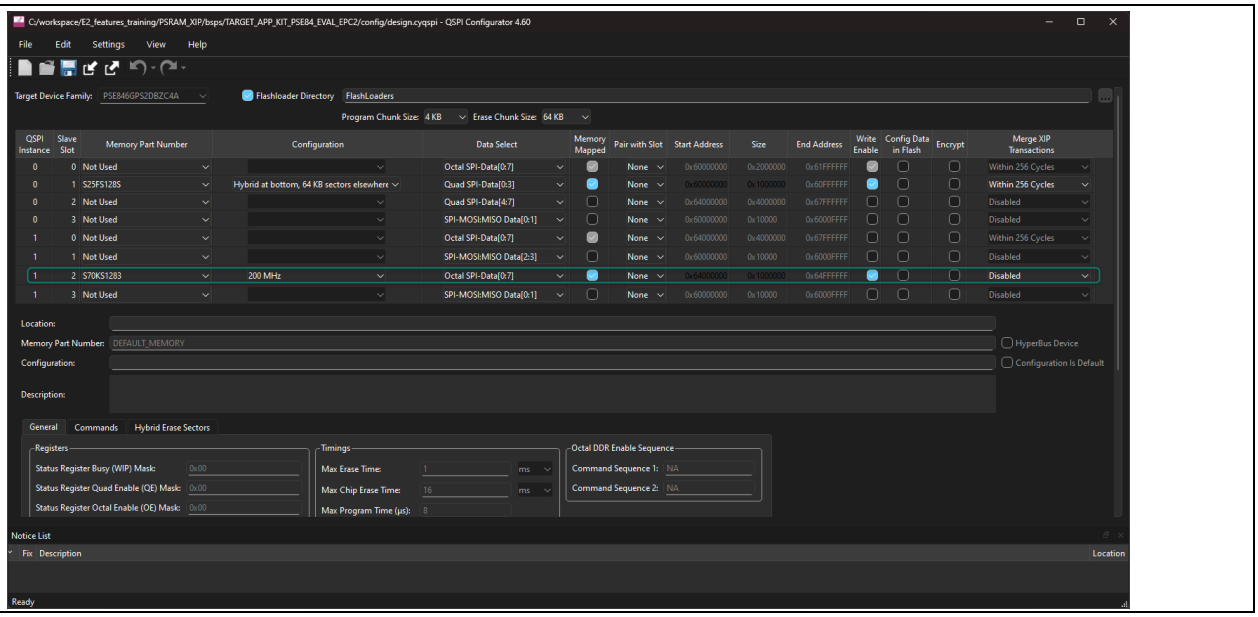


Figure 16 Octal PSRAM configuration in QSPI Configurator

Observe the following fields:

- **QSPI Instance**: This memory is using SMIF1
- **Slave slot**: Using SMIF1_select2
- **Memory Part Number**: Using the S70KS1283 Octal SPI PSRAM included in SOM
- **Configuration:** Memory is configured at 200MHz. Other frequencies are supported
- **Data select:** This memory is implemented in octal SPI configuration
- **Memory Mapped:** Enables XIP mode
- **Pair with Slot:** Dual quad operation is not used
- **Start Address/Size/End Address:** This memory is placed from 0x6400_0000-0x64FF_FFFF
- **Write Enable**: Write access is enabled
- **Config Data in Flash**: Configuration is placed in SRAM
- **Encrypt:** Encryption is not used
- **Merge XIP Transactions**: Transactions are not merged

4. From the Project Explorer window, expand the CM33 secure project and open the main.c file. “main.c” of the CM33_s project has application code for the PSRAM Read and Write in XIP.
5. Open *proj_cm33_s/main.c* to observe the code flow. The initialization code is in four parts, as follows:

   a) Initialize the SMIF1 using serial-flash and PDL APIs.

Code listing 6 Octal PSRAM initialization

```
static void smif_ospi_psram_init(void)
{
    cy_rslt_t result;

    /* Disable SMIF Block for reconfiguration. */
    Cy_SMIF_Disable(CYBSP_SMIF_CORE_1_PSRAM_HW);
```

Code example: PSRAM read and write in XIP mode

```
    /* Initialize SMIF-1 Peripheral. */
    result = Cy_SMIF_Init((CYBSP_SMIF_CORE_1_PSRAM_hal_config.base),
                          (CYBSP_SMIF_CORE_1_PSRAM_hal_config.config),
                          SMIF_INIT_TIMEOUT_USEC, &smif_mem_context.smif_context);

    check_status("Cy_SMIF_Init failed", result);

    /* Configure Data Select Option for SMIF-1 */
    Cy_SMIF_SetDataSelect(CYBSP_SMIF_CORE_1_PSRAM_hal_config.base,
                          smif1BlockConfig.memConfig[0]->slaveSelect,
                          smif1BlockConfig.memConfig[0]->dataSelect);

    /* Enable the SMIF_CORE_1 block. */
    Cy_SMIF_Enable(CYBSP_SMIF_CORE_1_PSRAM_hal_config.base,
&smif_mem_context.smif_context);

    /* Set-up serial memory. */
    result = mtb_serial_memory_setup(&serial_memory_obj,
                                MTB_SERIAL_MEMORY_CHIP_SELECT_2,
                                CYBSP_SMIF_CORE_1_PSRAM_hal_config.base,
                                CYBSP_SMIF_CORE_1_PSRAM_hal_config.clock,
                                &smif_mem_context,
                                &smif_mem_info,
                                &smif1BlockConfig);

    check_status("serial memory setup failed", result);
}
```

b) Configure SMIF1 to work in XIP mode (by default PDL configures SMIF in MMIO mode).

Code listing 7 PSRAM being initialized for XIP

```
/* Enable XIP mode for the SMIF memory slot associated with the PSRAM. */
result = mtb_serial_memory_enable_xip(&serial_memory_obj, true);
check_status("mtb_serial_memory_enable_xip: failed", result);
```

c) Send the command to enable write access

Code listing 8 PSRAM initialized for write access

```
/* Enable write for the SMIF memory slot associated with the PSRAM. */
result = mtb_serial_memory_set_write_enable(&serial_memory_obj, true);
check_status("mtb_serial_memory_set_write_enable: failed", result);
```

6. Build and program the application.

4.6 Output

After successful programming, the application starts automatically. Confirm that the UART terminal displays the following terminal output:

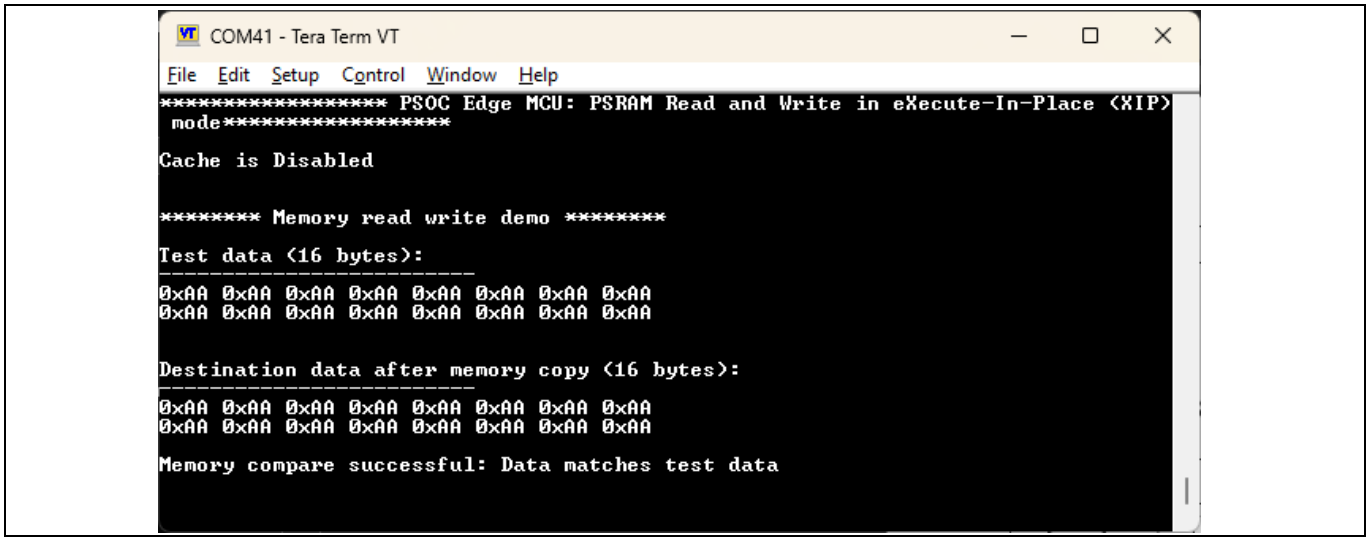


Figure 17 Output of PSRAM application

4.7 Modify the application

4.7.1 Enable caching

The default example is not enabling the SMIF1 cache.

We can enable the cache to make the SMIF1 region cacheable, and set the attributes to Write-Back and Read & Write Allocate to demonstrate cache clean and invalidation operations.

When SMIF cache is enabled, the memory read / write operations pass through the SMIF cache and follows the cache properties and rules.

- **Cache clean:** When writing into the target memory in XIP mode, the data will be written to the cache first. Since cache is configured with the Write-back attribute, the data in cache and the target memory (MMIO read) will be different as seen from the terminal output. When the cache clean operation is carried out, the data is transferred from the cache to the target memory.
- **Cache invalidate:** Data is written to the target memory address using MMIO mode. Upon reading the same address using the XIP mode, the data will still be different from what was written. When performing cache invalidation, the data from the target memory now moves to the cache. The same can be observed from the terminal output.

Hints for the advanced modification

- The project includes a macro to enable cache in the main file

Code example: PSRAM read and write in XIP mode

Solution

1. In *proj_cm33_s/main.c*, update the macro.

Code listing 9 Enabling cache for PSRAM

```
/*******************************************************************************
* Macros
*******************************************************************************/
#define CM33_NS_APP_BOOT_ADDR      (CYMEM_CM33_0_m33_nvm_START + \
                                    CYBSP_MCUBOOT_HEADER_SIZE)
#define CACHE_ENABLE                (1U)
#define ADDRESS_SIZE_IN_BYTES       (4U)
```

2. Rebuild and reprogram the application. Observe the cache clean and cache invalidate operations for the data written into the PSRAM with caching enabled.

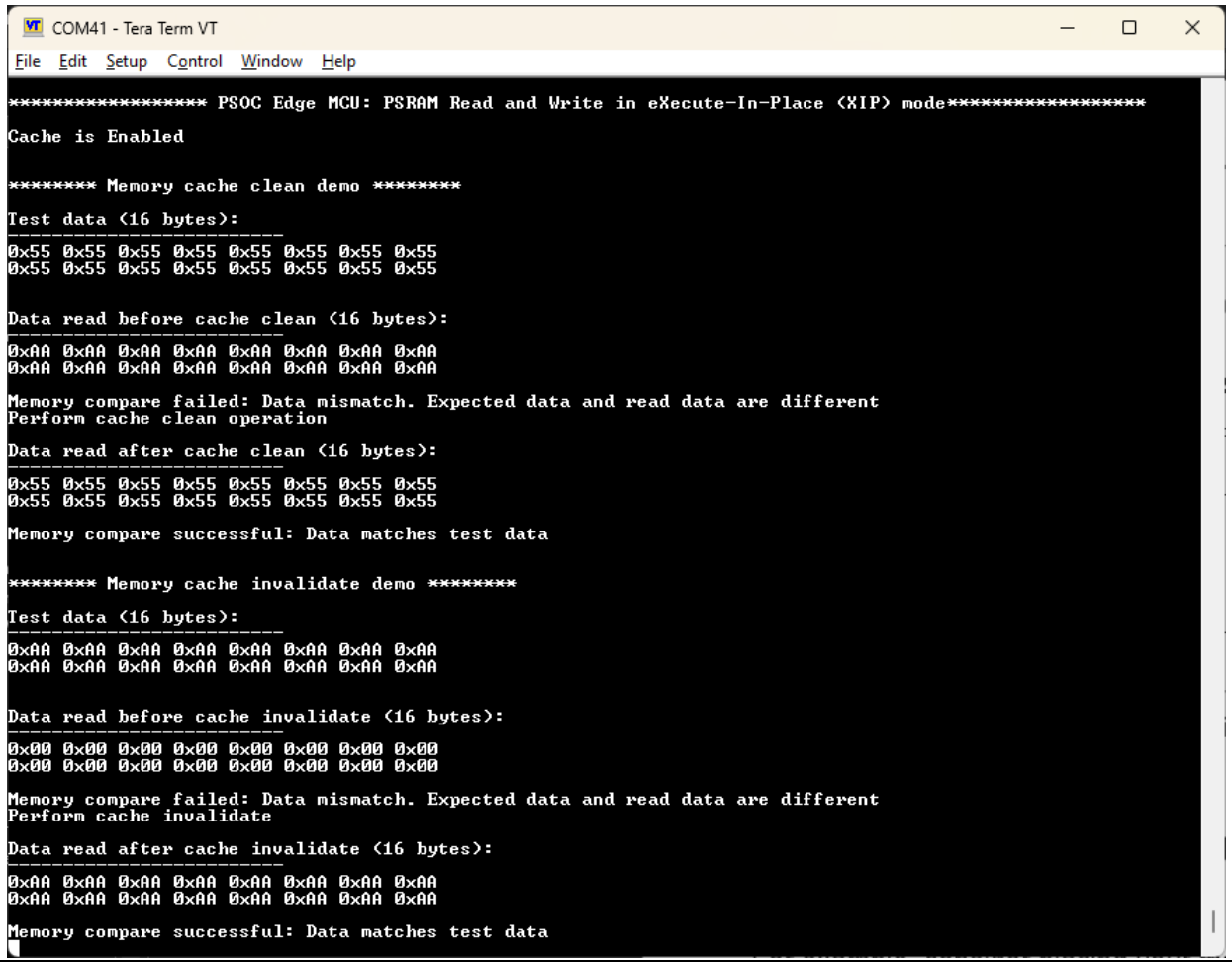


Figure 18 Output of PSRAM example after enabling cache

4.7.2 Placing an array to PSRAM

For our second modification, try modifying the application to place an array into the external PSRAM. This can be useful when requiring large chunks of data, such as a machine learning data model.

Hints for the advanced modification

- Use Memory Configurator to create a new section for data
- Modify the linker file to place data in the new section

Solution

1. Open **Device Configurator** from the Quick Panel, then select the **Memory** tab, and expand the **SMIF block 1, memory 2** corresponding to the PSRAM memory.

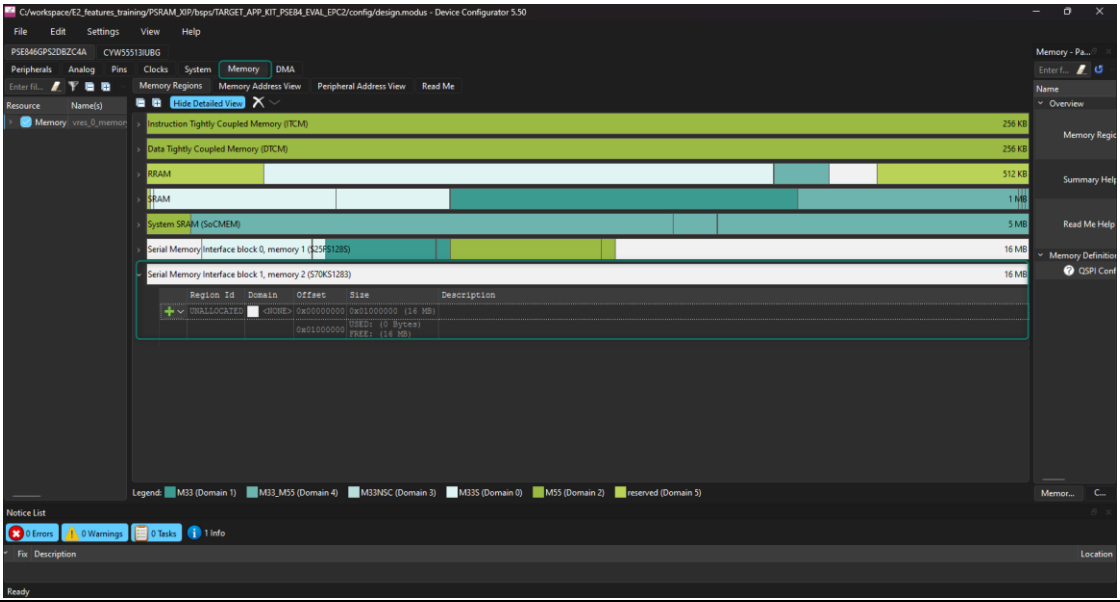


Figure 19 Default configuration of Memory for PSRAM example

2. Click the plus sign in the unallocated region and select “Add region”.
3. Type “data_psram” as **Region id**, select “M33S (Domain 0)” as the **Domain**, and select the **Offset** and **Size** to 0x0000000 and 0x01000000 respectively. This will create a custom 16MB region accessible from the CM33 secure application from 0x74000000-0x74FFFFFF.

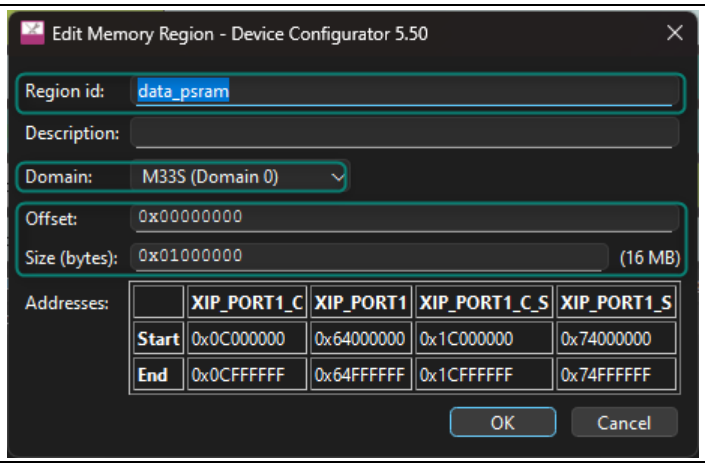


Figure 20 Creating new section in PSRAM

4. Click **OK** to save the new region. The Memory Regions will update as shown below:



Figure 21 New PSRAM section added to memory

5. Open the **Memory Address View** panel and select the **CM33 (secure) core** to observe the new section in the memory map.

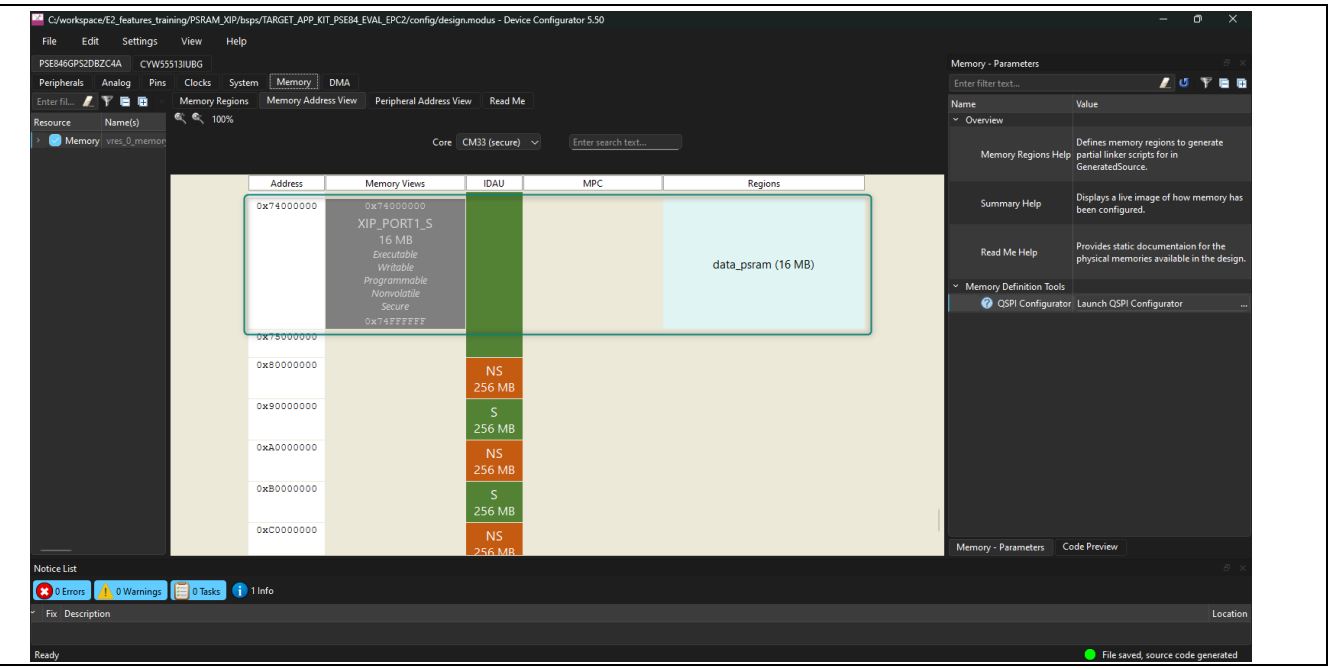


Figure 22 Memory address view showing new PSRAM section

6. Save and close the device configurator. This will generate updated files for the project.

7. Define new sections in the CM33_S linker file located at *<project_path>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\COMPONENT_CM33\TOOLCHAIN_GCC_ARM\pse84_s_cm33.ld*

   Note: Observe that we use the prefix "_S" to indicate that the memory is available in secure domain.

Code listing 10 Add new PSRAM section to linker file

```
/* Define the sections */
SECTIONS
{
    .psram_cm33_section (NOLOAD):
    {
        KEEP(*(.psram_cm33_section))
    } > data_psram_S

    /* This section reserves a space for MCUBoot header */
```

8. In *proj_cm33_s/main.c,* add the following code to declare an array in the new section:

Code listing 11 Declare a new array in the custom PSRAM section

```
/*******************************************************************************
 * Global Variables
 *******************************************************************************/
static mtb_serial_memory_t serial_memory_obj;
static cy_stc_smif_mem_context_t smif_mem_context;
static cy_stc_smif_mem_info_t smif_mem_info;
__attribute__((section(".psram_cm33_section"))) uint8_t data_array[100] = {0};
```

9. Then, add code to the main() function to view the array, modify it and read it back.

Code listing 12 Add code to use array in application

```
#if (1U == CACHE_ENABLE)
    memory_cache_clean_demo(SMIF_1_PSRAM_SECURE_ADDRESS, TEST_DATA_2,
NUM_BYTES_PER_LINE);
    memory_cache_invalidate_demo(SMIF_1_PSRAM_SECURE_ADDRESS, TEST_DATA_1,
NUM_BYTES_PER_LINE);
#else
    memory_read_write_demo(SMIF_1_PSRAM_SECURE_ADDRESS, TEST_DATA_1,
NUM_BYTES_PER_LINE);
#endif


    /****** Modification Start *****/
    printf("\n=========================\n\r");
    printf("Modification\n\r");
    printf("=========================\n\n\r");

    printf("\r\nAddress of data_array = %p\n\r", (void*)data_array);
    printf("data_array is populated with sequential numbers\n\r");
    for(int i=0; i<16; i++)
    {
        data_array[i]=i;
    }
    print_array("Read Data", data_array, 16);

    printf("\r\nAddress of data_array = %p\n\r", (void*)data_array);
    printf("Each element of data_array is incremented by 1\n\r");
    for(int i=0; i<16; i++)
    {
        data_array[i]++;
```

```
    }
    print_array("Read Data", data_array, 16);
    /****** Modification End *****/


    while (cy_retarget_io_is_tx_active());
```

10. Rebuild and reprogram the example. Observe that the array is placed in address 0x7400_0000, which corresponds to the secure area in PSRAM accessed using SMIF1.
    A data array of 16 bytes has been populated in that region. Then, it is modified by adding 1 to each value, demonstrating successful read and write functionality of PSRAM.

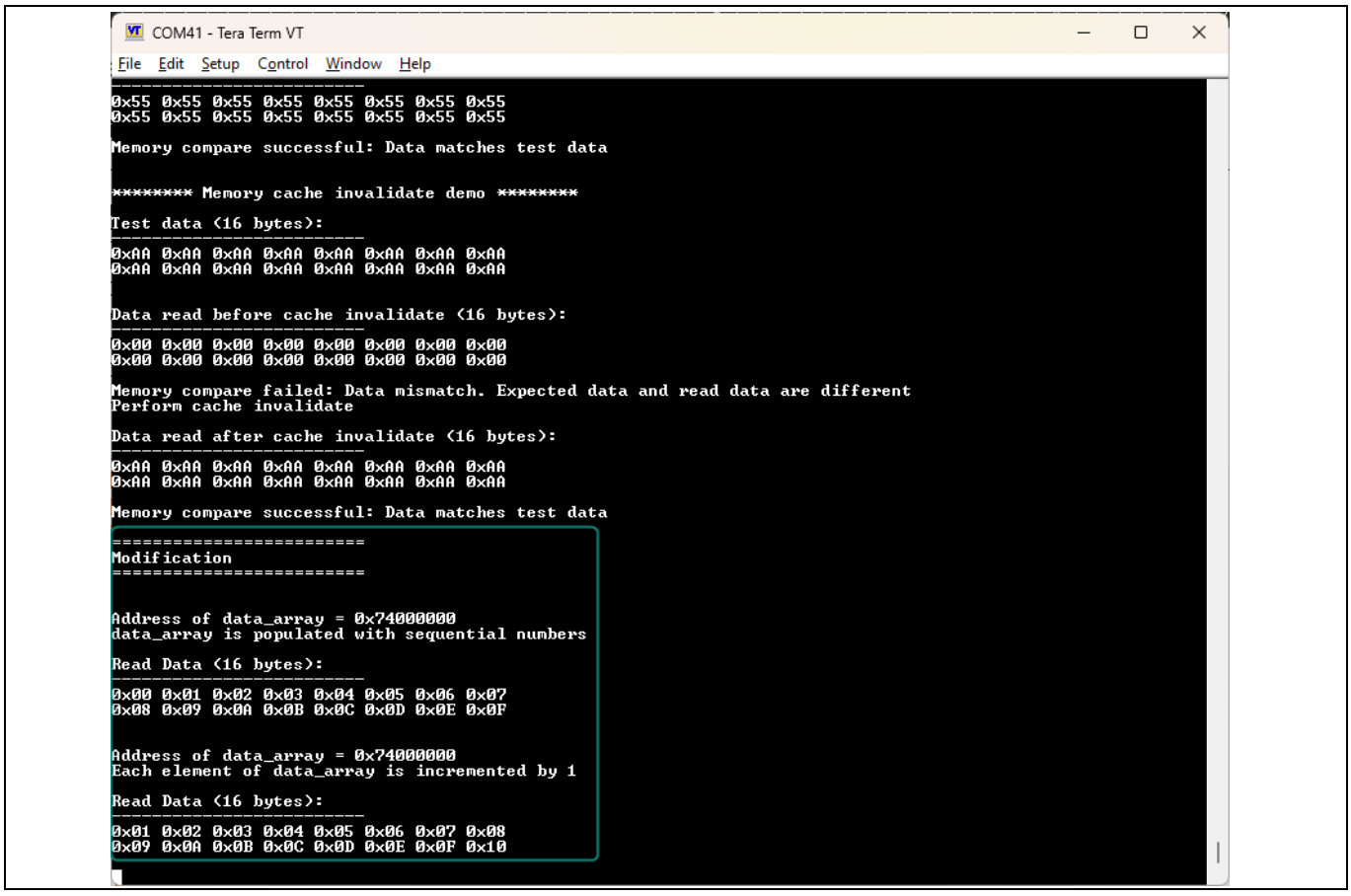


Figure 23 Output after defining a custom array in PSRAM

4.8 Conclusion

We successfully created an application to write and read PSRAM in XIP mode, using ModusToolbox™. We learned how to use SMIF block of the PSOC™ Edge E84 MCU to interface PSRAM memory, and how to read and write in XIP mode.

5 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**.

Note: If you have not installed ModusToolbox™ or Eclipse IDE, see the Prerequisites section.

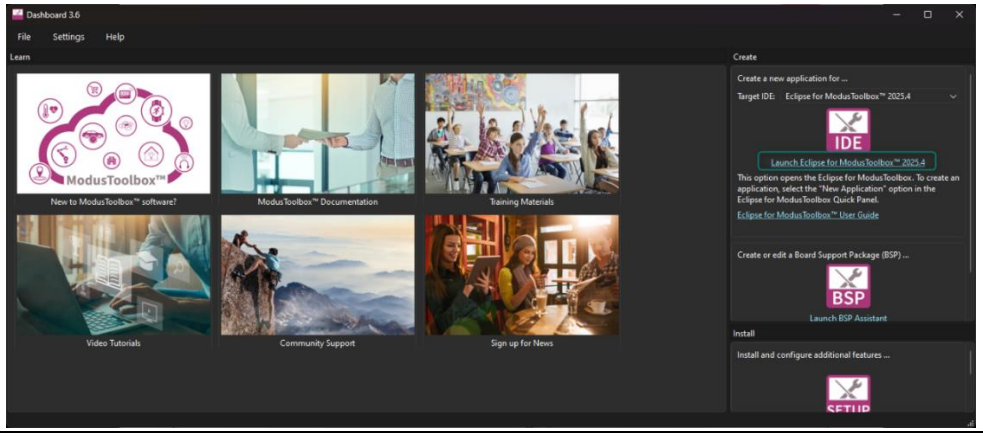


Figure 24 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project.
   A folder will be created if the workspace does not exist.
   After selecting a suitable directory, click the **Launch** button.
   Note: It is recommended to keep the workspace path short. Windows has a 260 characters path-length limit and a long workspace path may cause build issues when creating some applications.

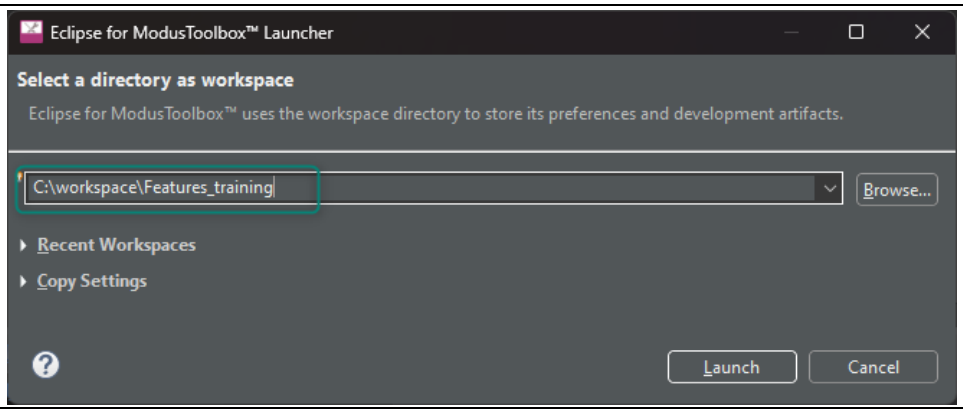


Figure 25 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**.

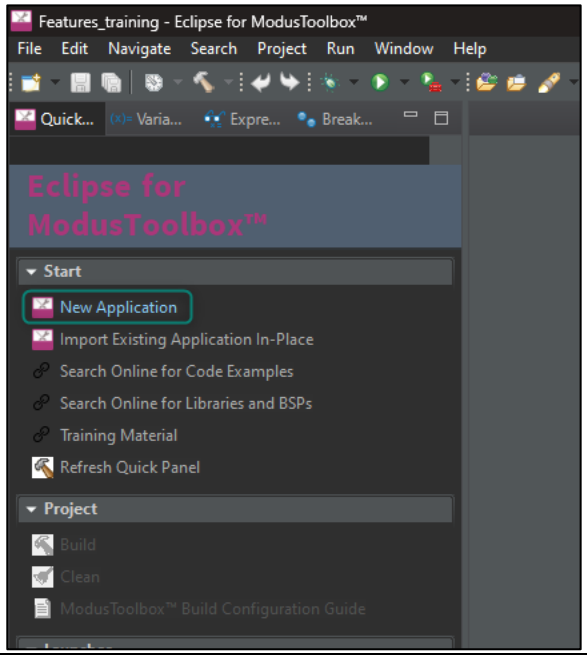


Figure 26 Create new application

4. Once **Projector Creator** launches, you will be prompted to select a board support package (BSP).
   Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next**.
   *Note: BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application.*
   Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL which includes EPC2 devices by default.

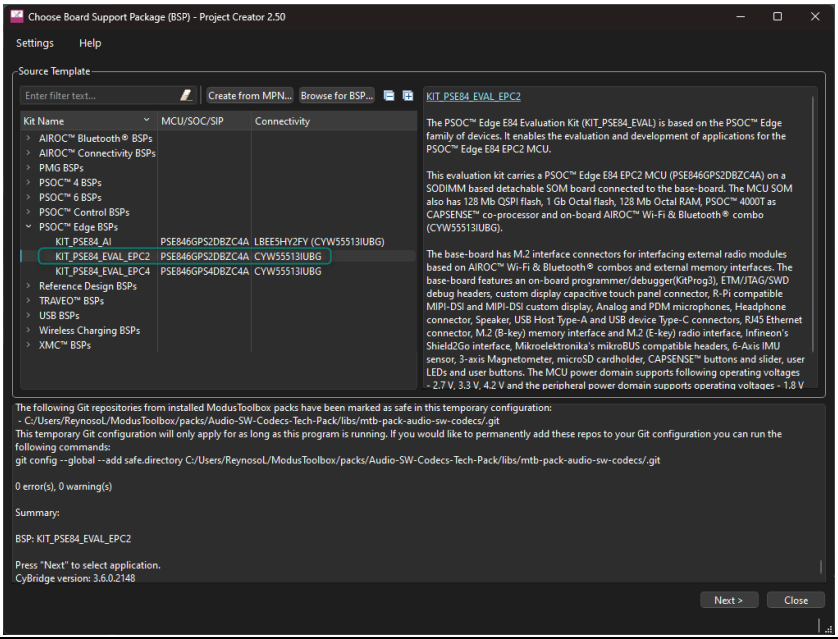


Figure 27 Selecting BSP in Project Creator

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and then click **Create**.

   Figure 28 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Note: Windows has a 260 characters path-length limit. A long workspace path and/or a long project name may cause build issues when creating some applications.

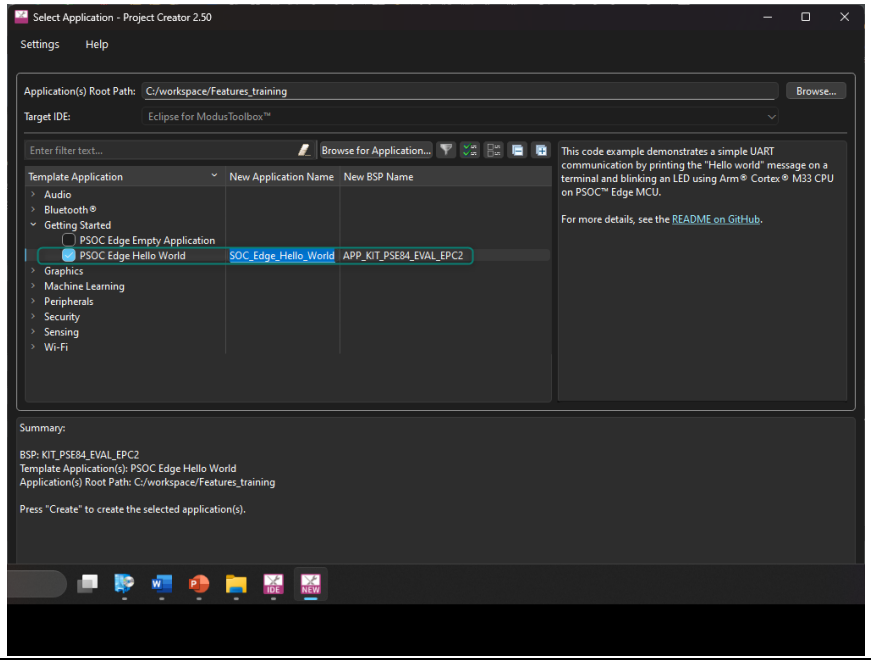


Figure 28 Selecting application

6. A new project is created, and with all the files visible in the **Project Explorer** window.

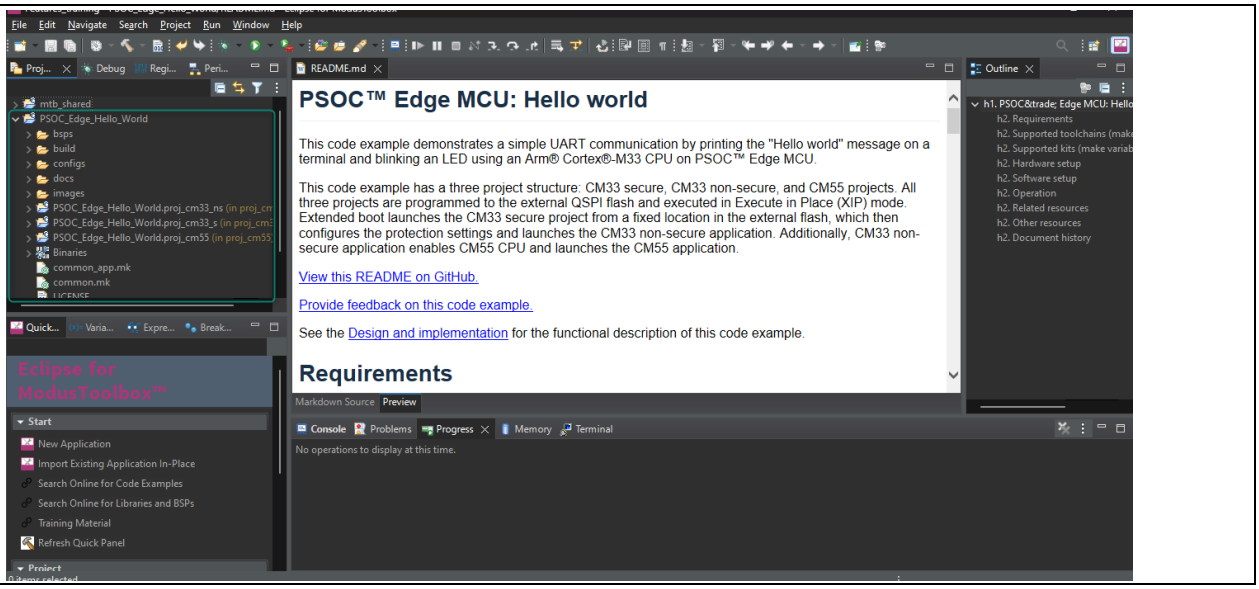


Figure 29 New application created

6 Appendix B: KIT_PSE84_EVAL details

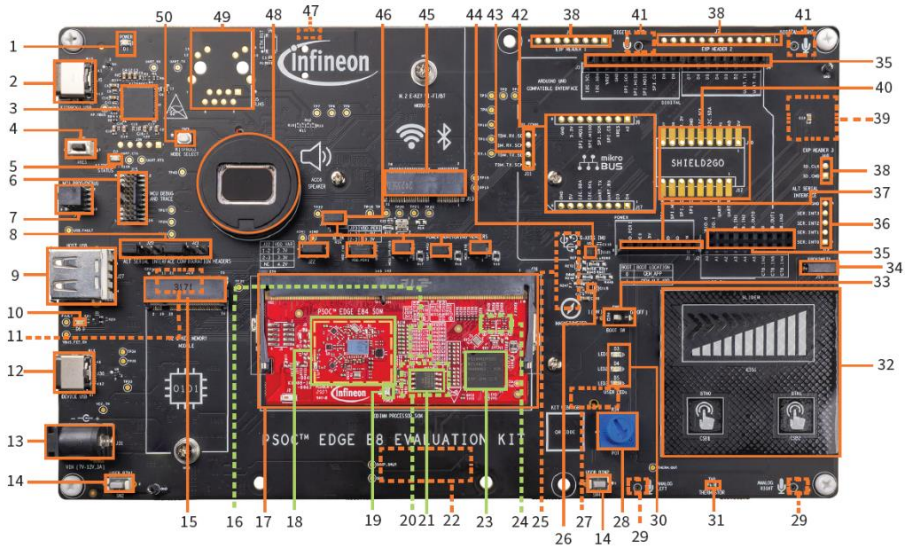


Figure 30 PSOC™ Edge evaluation kit (EVK) details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C Power Delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128-Mb Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor system-on-module (SoM) 260-pin SODIMM connector (J28)
18 AIROC™ CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor system-on-module (SoM) power LED (D3)
20 1-Gb Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128-Mb Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi-compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

* Footprint only, not populated on the board
** Component at the bottom side of the Baseboard
*** Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-01-15	Initial release.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

PSOC™, formerly known as PSoC™, is a trademark of Infineon Technologies. Any references to PSoC™ in this document or others shall be deemed to refer to PSOC™.

Edition 2026-01-15

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com